

Лабораторна робота №4.

Нейронні мережі та згорткові архітектури на CIFAR-10

26 серпня 2026 р.

Мета

Реалізувати багат шаровий перцептрон двічі — власноруч на NumPy зі своїм зворотним поширенням і той самий у PyTorch — та довести їхню чисельну еквівалентність; далі побудувати згорткову мережу на CIFAR-10 і виміряти, скільки насправді дає кожен інгредієнт сучасного рецепта: архітектура, ініціалізація, нормалізація та аугментація.

Теоретичні відомості

CIFAR-10 — 60 000 кольорових фотографій 32×32 у десяти класах. На відміну від рукописних цифр, тут об'єкт може бути під будь-яким кутом, на будь-якому тлі й будь-якого кольору, тому повнозв'язна мережа впирається приблизно в 50 % правильних відповідей, а різниця між нею та згортковою стає очевидною, а не косметичною.

- **Зворотне поширення.** Градієнт за всіма параметрами рахується одним проходом назад за ланцюговим правилом, тому коштує приблизно стільки ж, скільки прямий прохід. Реалізацію завжди перевіряють скінченними різницями: помилка в градієнті виглядає як «модель погано навчається» і легко приймається за невдалий темп навчання.
- **Активації та зникаючий градієнт.** Похідна сигмоїди не перевищує $1/4$, тож у мережі з десяти шарів множник сягає 10^{-6} і перші шари майже не отримують сигналу. ReLU має похідну 1 на додатній півосі й цієї проблеми не створює, натомість може «вимикати» нейрони назавжди.
- **Згортковий шар.** Замість зв'язку кожного входу з кожним виходом застосовується невелике ядро в кожній позиції. Кількість ваг $C_{out}(C_{in}K^2 + 1)$ не залежить від розміру зображення, а спільні ваги дають еквіваріантність до зсуву. Для входу $32 \times 32 \times 3$ економія проти повнозв'язного шару — два порядки.
- **Регуляризація глибоких мереж.** Пакетна нормалізація стабілізує масштаби активацій усередині мережі, dropout заважає нейронам покладатися на конкретних сусідів, аугментація породжує нові приклади перетвореннями, що зберігають мітку. На CIFAR-10 аугментація дає найбільший приріст із трьох, і це варто виміряти, а не прийняти на віру.

Корисні посилання для ознайомлення:

- [CIFAR-10](#) — опис набору; вантажиться через `torchvision.datasets.CIFAR10`;
- [Stanford CS231n](#) — конспекти лекцій із поясненням кожного шару; курс побудований саме навколо CIFAR-10;

- [Dive into Deep Learning](#) — підручник із виконуваним кодом на PyTorch;
- [PyTorch: Training a Classifier](#) — офіційний приклад саме на цьому наборі;
- [Google Colab](#) — безкоштовний GPU; на процесорі навчання займе десятки хвилин замість одиниць.

Порядок виконання

1. Завантажити CIFAR-10, показати приклади зображень і розподіл класів. Навести базову лінію — лінійну модель на сирих пікселях.
2. Реалізувати багатошаровий перцептрон на NumPy разом із власним зворотним поширенням, перевірити градієнт скінченими різницями та довести чисельну еквівалентність із тією самою мережею, зібраною засобами фреймворку.
3. Порівняти щонайменше чотири конфігурації «активація × ініціалізація» за однакових умов і пояснити, яка навчається найшвидше.
4. Спроектувати згорткову мережу, порахувавши кількість параметрів кожного шару вручну до запуску. Порівняти її з повнозв'язною за кількістю ваг, часом епохи й точністю.
5. Виміряти внесок пакетної нормалізації та dropout окремо й разом. Результати навести таблицею.
6. Порівняти помірну й надмірну аугментацію з навчанням без неї. Пояснити, які перетворення для цього набору зберігають мітку, а які — ні.
7. Побудувати криві навчання для двох розмірів тренувальної вибірки й матрицю похибок найкращої моделі. Знайти пару класів, які плутаються найчастіше, і пояснити чому.
8. Звести всі моделі в таблицю «параметри — час епохи — точність — приріст до базової лінії» та зробити висновок, який прийом дав найбільше.

Додаткові завдання (необов'язкові)

Ці пункти виходять за межі аудиторного часу й виконуються за бажанням; вони не впливають на основну оцінку, але зараховуються як бонус.

1. Розширити пункт 5 до повної сітки: чотири активації (`sigmoid`, `tanh`, `relu`, `gelu`) на двох ініціалізаціях — вісім конфігурацій на одному графіку.
2. Додати до згорткової мережі залишкові з'єднання (один блок `ResidualBlock`) і порівняти збіжність із мережею без них за однакової глибини.
3. Узяти `resnet18` із `torchvision.models` з попередньо навченими вагами, донавчити останній шар на CIFAR-10 і порівняти з власною мережею за точністю та часом.

Контрольні запитання

1. Що станеться з багатошаровою мережею, якщо активацію замінити тотожною функцією?
2. Ви реалізували зворотне поширення, і модель не навчається. Як за одну перевірку відрізнити помилку в градієнті від невдалого темпу навчання?
3. Похідна сигмоїди не перевищує $1/4$. Яке число це дає для мережі з десяти шарів і що з цього випливає?
4. Скільки параметрів має згортковий шар із 64 фільтрами 3×3 над входом із 32 каналів, і чому це число не залежить від розміру зображення?
5. Чому на CIFAR-10 різниця між повнозв'язною та згортковою мережею набагато більша, ніж на рукописних цифрах?
6. Горизонтальне віддзеркалення для CIFAR-10 доречне, а вертикальне — ні. Чому, і на якому наборі було б навпаки?
7. Мережа дала 75 % на CIFAR-10. Чого ще бракує, щоб назвати результат хорошим?